



ΧΑΡΟΚΟΠΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΜΑΤΙΚΗΣ

ΣΧΟΛΗ: Ψηφιακής Τεχνολογίας

ΤΜΗΜΑ: Πληροφορικής και Τηλεματικής

ΠΡΟΓΡΑΜΜΑ ΜΕΤΑΠΤΥΧΙΑΚΩΝ ΣΠΟΥΔΩΝ: Προηγμένες
Τεχνολογίες Πληροφορικής και Εφαρμογές

ΚΑΤΕΥΘΥΝΣΗ: Επιστήμη των Δεδομένων και Τεχνητή
Νοημοσύνη (1η κατεύθυνση)

Τίτλος Εργασίας

**Logistics Καυσίμων και Safety Monitor Platform: Cloud Native
Architecture**

Ευφροσύνη Κούμπουλη (Α.Μ. 25108)

Αθήνα, 2026

ΠΕΡΙΕΧΟΜΕΝΑ

Περίληψη

1. Εισαγωγή & Περιγραφή Επιχειρησιακού Προβλήματος
 - 1.1. Το Επιχειρησιακό Πρόβλημα & οι Περιορισμοί Ασφαλείας
 - 1.2. Σενάριο Χρήσης & Ιεραρχική Οργάνωση IoT Υποδομής
2. Αρχιτεκτονική Συστήματος & Μοτίβα Σχεδιασμού (Design Patterns)
 - 2.1. Μετασχηματισμός Υποδομής: Από το Docker Compose στο Κατανεμημένο Kubernetes (MicroK8s)
 - 2.2. Εφαρμογή Μοτίβων Σχεδιασμού (Cloud Design Patterns)
 - 2.2.1. Μοτίβο Απουσίας Κατάστασης (Stateless Pattern)
 - 2.2.2. Μοτίβο Ασύγχρονης Απάντησης / Μηνυμάτων (Asynchronous Response / Messaging Pattern)
 - 2.2.3. Υλοποίηση Web UI Monitor μέσω Stateless Deployment & Kubernetes ConfigMap
3. Αυτοματοποίηση (Automation) & GitOps
 - 3.1. Υποδομή ως Κώδικας (Infrastructure as Code - IaC) μέσω OpenTofu & Ansible
 - 3.2. Το GitOps Pipeline: Jenkins (CI/CD) & ArgoCD
 - 3.2.1. Continuous Integration (CI) μέσω Jenkins
 - 3.2.2. Continuous Deployment (CD) μέσω ArgoCD
4. Serverless Επεξεργασία (Serverless Computation)
 - 4.1. Θεωρητικό Υπόβαθρο & Αρχιτεκτονική Event-Driven
 - 4.2. Μεθοδολογία Υλοποίησης, Προκλήσεις και Εναλλακτική Προσέγγιση
5. Δοκιμές Φόρτου, Πειραματικές Μετρήσεις & SLAs (Service Level Agreements)
 - 5.1. Μεθοδολογία Πειραματικών Δοκιμών (Load Testing)
 - 5.2. Στατιστική Ανάλυση & Αθροιστική Συνάρτηση Κατανομής (CDF Percentiles)
 - 5.3. Καθορισμός Κλάσεων Ποιότητας Υπηρεσίας (SLA Levels)
6. Περιορισμοί Υλοποίησης & Μελλοντικές Βελτιώσεις (Future Work)
 - 6.1. Υφιστάμενοι Περιορισμοί του Πρωτοτύπου
 - 6.2. Μελλοντικές Επεκτάσεις για μια Παραγωγική Cloud Native Εφαρμογή (Production-Ready)
 - 6.2.1. Live Ενσωμάτωση Multi-Cloud IaC & Serverless Runtimes
 - 6.2.2. Διαχείρισης Κατάστασης & Ασφάλειας (State & Secret Management)
 - 6.2.3. Παρακολούθηση Μετρικών & Αυτόματη Κλιμάκωση (Observability & Autoscaling)

Περίληψη & Εισαγωγή

Η παρούσα εργασία αποτελεί την αρχιτεκτονική εξέλιξη και τον πλήρη μετασχηματισμό της πλατφόρμας παρακολούθησης logistics καυσίμων που σχεδιάστηκε αρχικά στο πλαίσιο του μαθήματος "Πλατφόρμες Νέφους". Η αρχική υλοποίηση βασίστηκε σε ένα μονολιθικό τοπικό περιβάλλον ενορχήστρωσης Docker Compose, το οποίο, αν και κάλυπτε τη βασική επιχειρηματική λογική συλλογής τηλεμετρίας και διαχείρισης συναγερμών μέσω ThingsBoard, RabbitMQ, Node-RED και MinIO, παρουσίαζε περιορισμούς ως προς την οριζόντια κλιμάκωση (scalability), την αυτόματη ανάκαμψη (fault tolerance) και την αυτοματοποιημένη διαχείριση της υποδομής.

Στο πλαίσιο των Cloud Native προδιαγραφών, η εφαρμογή αναδιαρθρώθηκε εκ βάθρων. Η συνολική υποδομή μεταφέρθηκε σε ένα κατακευματισμένο περιβάλλον ενορχήστρωσης Kubernetes (MicroK8s), οργανωμένο ως ένα Git Monorepo. Για τη βελτιστοποίηση της αρχιτεκτονικής, εφαρμόστηκαν προηγμένα cloud design patterns, όπως το Stateless Pattern για τα compute components και το Asynchronous Response Pattern μέσω message-driven ουρών. Παράλληλα, ενσωματώθηκε Serverless Computation (Knative) για event-driven triggered επεξεργασία δεδομένων και ελαχιστοποίηση της άσκοπης κατανάλωσης πόρων. Η διαχείριση του κύκλου ζωής της εφαρμογής βασίστηκε σε ένα πλήρως αυτοματοποιημένο GitOps Pipeline, συνδυάζοντας το Jenkins (CI/CD) για το automated container build και το ArgoCD για declarative συγχρονισμό της κατάστασης του cluster. Τέλος, η σταθερότητα και η ποιότητα παροχής υπηρεσιών (QoS) του συστήματος επικυρώθηκαν μέσω πειραματικών δοκιμών παραγωγής φόρτου (Load Testing) και στατιστικής ανάλυσης των χρόνων απόκρισης (95th και 99th CDF Percentiles) για τη διαμόρφωση συγκεκριμένων επιπέδων SLA (Service Level Agreements).

1. Εισαγωγή & Περιγραφή Επιχειρησιακού Προβλήματος

1.1. Το Επιχειρησιακό Πρόβλημα & οι Περιορισμοί Ασφαλείας

Η διαχείριση της εφοδιαστικής αλυσίδας και της διανομής πετρελαιοειδών και ενεργειακών προϊόντων χαρακτηρίζεται από εξαιρετικά υψηλό βαθμό επικινδυνότητας. Λόγω της φύσης των μεταφερόμενων υλικών (εύφλεκτα και εκρηκτικά υλικά), τυχόν αστοχίες στις υποδομές αποθήκευσης ή μεταφοράς μπορούν να επιφέρουν καταστροφικές συνέπειες για την ανθρώπινη ασφάλεια, το περιβάλλον και την επιχειρησιακή συνέχεια.

Για την αποτροπή τέτοιων συμβάντων, οι σύγχρονες βιομηχανίες εξοπλίζουν τις υποδομές τους με αισθητήρες Internet of Things (IoT), οι οποίοι καταγράφουν κρίσιμα φυσικά μεγέθη (θερμοκρασία, πίεση, στάθμη καυσίμου, κραδασμούς). Οι συσκευές αυτές παράγουν συνεχή ρεύματα τηλεμετρίας (telemetry streams) καθώς και σήματα συναγερμών (alarms) όταν παραβιάζονται τα όρια ασφαλείας. Το βασικό τεχνικό πρόβλημα μετατοπίζεται, συνεπώς, στη δημιουργία μιας cloud πλατφόρμας η οποία θα είναι σε θέση να:

- ✓ Καταναλώνει μαζικά και ασύγχρονα τα δεδομένα από εκατοντάδες γεωγραφικά κατανεμημένες συσκευές χωρίς απώλειες.
- ✓ Ανιχνεύει, να επεξεργάζεται και να δρομολογεί alarms σε πραγματικό χρόνο με εξαιρετικά χαμηλή υστέρηση (low latency).
- ✓ Αποθηκεύει τα ιστορικά γεγονότα σε ανθεκτικές, οικονομικά αποδοτικές και απομονωμένες δομές (Object Storage) για σκοπούς ανιχνευσιμότητας (audit logging) και μελλοντικής ανάλυσης.

1.2. Σενάριο Χρήσης & Ιεραρχική Οργάνωση IoT Υποδομής

Για την πλήρη προσομοίωση ενός ρεαλιστικού παραγωγικού περιβάλλοντος, το σενάριο χρήσης της πλατφόρμας οργανώθηκε με αυστηρή ιεραρχική δομή και multi-tenancy λογική. Το σύστημα διαχειρίζεται και επιτηρεί συνολικά 15 IoT συσκευές, οι οποίες κατανέμονται ισομερώς σε 3 διαφορετικούς πελάτες (Customers / Gas Stations), εξασφαλίζοντας πλήρη λογική απομόνωση των δεδομένων τους:

Gas Station 01 (Customer 01):

diesel truck 01: Αισθητήρες βυτιοφόρου μεταφοράς πετρελαίου (Τηλεμετρία: Στάθμη, Alarms: LOW_FUEL).

gas truck 01: Αισθητήρες βυτιοφόρου μεταφοράς βενζίνης (Alarms: HIGH_PRESSURE).

diesel tank 01: Αισθητήρες σταθερής δεξαμενής πετρελαίου (Τηλεμετρία: Όγκος, Θερμοκρασία).

gas tank 01: Αισθητήρες σταθερής δεξαμενής βενζίνης (Alarms: HIGH_TEMPERATURE).

warehouse 01: Αισθητήρες περιβάλλοντος κεντρικής αποθήκης ελαιολιπαντικών και ξυλείας.

Gas Station 02 και 03 (Customer 02 & Customer 03):

Περιλαμβάνουν τις αντίστοιχες 5 συσκευές επιτήρησης (diesel truck 02, gas truck 02, diesel tank 02, gas tank 02, warehouse 02) με αυτόνομα Access Tokens επικοινωνίας.

Κάθε συσκευή πραγματοποιεί αυθεντικοποίηση στην πλατφόρμα μέσω μοναδικού κρυπτογραφικού Access Token. Όταν μια συσκευή αποστέλλει τηλεμετρία που ξεπερνά τα όρια ασφαλείας, η IoT πλατφόρμα εκκινεί μια αλυσίδα επεξεργασίας γεγονότων (event-driven chain). Ο παραγόμενος συναγερμός εμπλουτίζεται δυναμικά, μετατρέπεται σε δομημένο έγγραφο JSON και αποθηκεύεται στο MinIO Object Storage, σε διακριτά buckets που αντιστοιχούν αποκλειστικά στον πελάτη (Gas Station) στον οποίο ανήκει η συσκευή.

2. Αρχιτεκτονική Συστήματος & Μοτίβα Σχεδιασμού (Design Patterns)

2.1. Μετασχηματισμός Υποδομής: Από το Docker Compose στο Κατανεμημένο Kubernetes (MicroK8s)

Η αρχική υλοποίηση του συστήματος βασιζόταν σε ένα flat περιβάλλον ενορχήστρωσης Docker Compose, όπου όλα τα components (ThingsBoard, RabbitMQ, Node-RED, MinIO) εκτελούνταν ως μεμονωμένα containers πάνω σε έναν κοινό πυρήνα και δίκτυο του ίδιου host. Αν και η προσέγγιση αυτή ήταν επαρκής για τη φάση της αρχικής ανάπτυξης (development prototype), εμφάνιζε εγγενείς αδυναμίες:

Έλλειψη Elasticity: Δεν υπήρχε δυνατότητα οριζόντιας κλιμάκωσης (horizontal scaling) μεμονωμένων υπηρεσιών (π.χ. scale-up μόνο του Node-RED όταν αυξάνονται τα alarms).

Χειροκίνητο Fault Tolerance: Σε περίπτωση αποτυχίας ενός container, η ανάκαμψη βασιζόταν αποκλειστικά στις τοπικές οδηγίες restart: always του Docker, χωρίς τη δυνατότητα orchestration επιπέδου cluster.

Για την επίλυση των παραπάνω, η αρχιτεκτονική μετασχηματίστηκε ριζικά με τη μεταφορά της σε ένα cloud-native περιβάλλον MicroK8s (Kubernetes). Το σύστημα πλέον οργανώνεται σε Pod Deployments, όπου η διαχείριση του δικτύου, η δρομολόγηση, η απομόνωση των πόρων και η διαθεσιμότητα ελέγχονται από το Kubernetes Control Plane.

Η φυσική και λογική κατανομή των microservices στο cluster υλοποιήθηκε ως εξής:

- *ThingsBoard Pod (thingsboard deployment)*: Λειτουργεί ως η πύλη εισόδου (IoT Gateway) για τη συλλογή τηλεμετρίας και τη διαχείριση των 15 συσκευών μέσω HTTP/MQTT API, εκτεθειμένο εσωτερικά στο cluster μέσω ClusterIP Service.

- *RabbitMQ Pod (rabbitmq deployment)*: Λειτουργεί ως ο κεντρικός Message Broker. Διαχειρίζεται τις ασύγχρονες ουρές μηνυμάτων (Message Queues) για τη διασφάλιση της ανθεκτικότητας των alarms.

- *Node-RED Pod (node-red deployment)*: Λειτουργεί ως η μηχανή επεξεργασίας και ενορχήστρωσης των γεγονότων (Event Choreography Engine), καταναλώνοντας μηνύματα από το RabbitMQ και αλληλεπιδρώντας με το MinIO.

- *MinIO Pod (minio deployment)*: Παρέχει την υποδομή Object Storage, οργανωμένη σε buckets, για την τελική αποθήκευση των JSON logs.

2.2. Εφαρμογή Μοτίβων Σχεδιασμού (Cloud Design Patterns)

Σύμφωνα με τις προδιαγραφές σχεδίασης αξιόπιστων cloud-native εφαρμογών, ενσωματώθηκαν δύο θεμελιώδη αρχιτεκτονικά μοτίβα:

2.2.1. Μοτίβο Απουσίας Κατάστασης (Stateless Pattern)

Μία από τις θεμελιώδεις αρχές και βασικές προδιαγραφές της σύγχρονης Cloud Native αρχιτεκτονικής είναι ο πλήρης διαχωρισμός της υπολογιστικής ισχύος (computation/processing) από την κατάσταση των δεδομένων (state). Στην παρούσα υποδομή, τα Pods που αντιστοιχούν στα υποσυστήματα Node-RED, ThingsBoard και RabbitMQ σχεδιάστηκαν, παραμετροποιήθηκαν και αναπτύχθηκαν ως Stateless Deployments.

➤ *Τεχνική Σκοπιμότητα*: Τα συγκεκριμένα Pods δεν αποθηκεύουν κανένα κρίσιμο δεδομένο ή μεταβαλλόμενη κατάσταση στο τοπικό, εφήμερο σύστημα αρχείων τους (ephemeral file system). Σε περίπτωση που ένα Pod τερματιστεί απροσδόκητα ή επανεκκινηθεί από το Kubernetes Control Plane (όπως επιβεβαιώθηκε κατά τη φάση των δοκιμών ανθεκτικότητας), ένα νέο πανομοιότυπο Pod δημιουργείται ακαριαία μέσω των μηχανισμών αυτοϊασης (self-healing), χωρίς να προκαλείται καμία αλλοίωση στην ακεραιότητα των δεδομένων ή στη συνολική λειτουργία του συστήματος.

➤ *Αρχιτεκτονική Αμυνα (Production Considerations)*: Η υιοθέτηση της stateless προσέγγισης για τα compute pods εφαρμόστηκε σκόπιμα με στόχο τη μεγιστοποίηση του throughput και την ελαχιστοποίηση των καθυστερήσεων κατά τη φάση των load tests. Σημειώνεται, ωστόσο, ότι για τη μετάβαση της πλατφόρμας σε ένα πλήρες παραγωγικό περιβάλλον (Production Environment), η αρχιτεκτονική σχεδίαση προβλέπει τη μετατροπή των ThingsBoard και RabbitMQ σε StatefulSets. Τα StatefulSets αυτά θα συνδέονται με μόνιμους αποθηκευτικούς πόρους μέσω Persistent Volumes (PVs) και Persistent Volume Claims (PVCs), διασφαλίζοντας ότι η εσωτερική κατάσταση, οι ρυθμίσεις και οι ουρές μηνυμάτων θα διατηρούνται μόνιμα σε εξωτερικά block storages του cloud.

2.2.2. Μοτίβο Ασύγχρονης Απάντησης / Μηνυμάτων (Asynchronous Response / Messaging Pattern)

Για την αποφυγή της στενής σύζευξης (tight coupling) και των blocking καταστάσεων μεταξύ του IoT Gateway (ThingsBoard) και της μηχανής επεξεργασίας (Node-RED), εφαρμόστηκε το Message-driven Asynchronous Pattern μέσω του RabbitMQ Broker.

Λειτουργία: Όταν μια συσκευή (π.χ. το gas truck 01) στέλνει ένα alarm, το ThingsBoard δεν περιμένει να ολοκληρωθεί η επεξεργασία του και η αποθήκευση στο MinIO για να απαντήσει στη συσκευή. Αντίθετα, δημοσιεύει (publishes) ακαριαία το alarm σε ένα Exchange του RabbitMQ και επιστρέφει άμεσα επιβεβαίωση λήψης (HTTP 200 OK) στη συσκευή.

Πλεονέκτημα (Queue Buffering): Το μήνυμα αποθηκεύεται με ασφάλεια σε μια ουρά (queue). Το Node-RED καταναλώνει ασύγχρονα (asynchronously consumes) τα μηνύματα από την ουρά με τον δικό του ρυθμό. Με αυτόν τον τρόπο, αν υπάρξει απότομη αιχμή φόρτου (traffic spike) από εκατοντάδες ταυτόχρονα alarms, η ουρά λειτουργεί ως buffer (αποσβεστήρας πίεσης), αποτρέποντας την κατάρρευση των backend υπηρεσιών και διασφαλίζοντας τη σταθερότητα της πλατφόρμας.

2.2.3. Υλοποίηση Web UI Monitor μέσω Stateless Deployment & Kubernetes ConfigMap

Για την οπτικοποίηση των κρίσιμων συμβάντων και των ιστορικών συναγερμών (alarms) σε πραγματικό χρόνο, σχεδιάστηκε και αναπτύχθηκε μια lightweight Frontend Web εφαρμογή σε περιβάλλον Python (Flask). Για την πλήρη εναρμόνιση με το Stateless Design Pattern και την αποφυγή permission constraints κατά το runtime, ο πηγαίος κώδικας της εφαρμογής ενσωματώθηκε declarative ως Kubernetes ConfigMap (frontend-source). Το Deployment του Frontend χρησιμοποιεί ένα επίσημο, minimal image (python:3.10-slim), το οποίο κατά την εκκίνηση πραγματοποιεί mount το ConfigMap ως volume στη διαδρομή /app και εκτελεί δυναμικά τις απαραίτητες εξαρτήσεις (pip install flask minio). Η εφαρμογή επικοινωνεί εσωτερικά στο cluster με το core MinIO Service (στη θύρα 9000), αντλεί ασύγχρονα τα JSON payloads από τα multi-tenancy buckets των 3 σταθμών (gas-station-01, gas-station-02, gas-station-03) και τα παρουσιάζει δομημένα σε ένα responsive Bootstrap UI με dark-theme παραμετροποίηση. Η υπηρεσία εκτίθεται εκτός cluster μέσω ενός NodePort Service στην προκαθορισμένη στατική πόρτα 30500.

3. Αυτοματοποίηση (Automation) & GitOps

3.1. Υποδομή ως Κώδικας (Infrastructure as Code - IaC) μέσω OpenTofu & Ansible

Στο πλαίσιο των Cloud Native προδιαγραφών για τη μέγιστη αυτοματοποίηση κατά τη φάση δημιουργίας και παραμετροποίησης των υπολογιστικών πόρων, η σχεδίαση της πλατφόρμας βασίζεται στη declarative προσέγγιση «Infrastructure as Code» (IaC). Για τον σκοπό αυτό, δημιουργήθηκε ο εξειδικευμένος κατάλογος automation/ στη ρίζα του κεντρικού Git Monorepo, ο οποίος περιλαμβάνει τα απαραίτητα αρχεία αρχιτεκτονικής διαμόρφωσης:

- Ορχήστρωση Υποδομής με OpenTofu (main.tf): Το OpenTofu χρησιμοποιείται για την αυτοματοποιημένη διαχείριση και παροχή (provisioning) των πόρων του δικτύου και των εικονικών μηχανών (Virtual Machines) σε περιβάλλον Public Cloud. Στο αρχείο main.tf δηλώνεται ο Cloud Provider (AWS) και καθορίζεται η δημιουργία ενός υπολογιστικού πόρου (EC2 Instance) με λειτουργικό σύστημα Ubuntu Server, το οποίο προορίζεται να λειτουργήσει ως ο κεντρικός κόμβος (node) του Kubernetes cluster.

- Διαχείριση Διαμόρφωσης με Ansible (playbook.yml): Μόλις ολοκληρωθεί η παροχή της εικονικής μηχανής, το Ansible αναλαμβάνει το Configuration Management. Το playbook playbook.yml είναι σχεδιασμένο να αποκτά δικαιώματα διαχειριστή (become: true) και να εκτελεί αυτόματα την εγκατάσταση και την αρχική παραμετροποίηση του MicroK8s (έκδοση 1.28/stable) μέσω του snap package manager, εξαλείφοντας την ανάγκη για χειροκίνητες εντολές στο τερματικό.

Προκλήσεις Υλοποίησης & Εναλλακτική Στρατηγική: Κατά την αρχική φάση του έργου, επιχειρήθηκε η ζωντανή (live) εκτέλεση των OpenTofu και Ansible για την αυτοματοποιημένη δημιουργία της υποδομής στον Cloud Provider. Ωστόσο, λόγω τεχνικών προβλημάτων και περιορισμών που παρουσιάστηκαν στο περιβάλλον υλοποίησης, η ζωντανή εκτέλεση και εφαρμογή των playbooks δεν κατέστη δυνατή.

Για τον λόγο αυτό, επιλέχθηκε ως εναλλακτική στρατηγική η declarative συγγραφή των manifests (main.tf και playbook.yml) και η ένταξή τους στον κατάλογο automation/ του Git Monorepo. Η προσέγγιση αυτή διασφαλίζει πλήρως τις αρχές του Infrastructure as Code, καθώς η υποδομή περιγράφεται λεπτομερώς ως κώδικας, είναι ελέγξιμη μέσω του GitOps pipeline και παραμένει έκδοση-έτοιμη (Deployment-Ready) για μελλοντική παραγωγική χρήση.

3.2. Το GitOps Pipeline: Jenkins (CI/CD) & ArgoCD

Η διαχείριση του κύκλου ζωής της εφαρμογής, η διανομή των container images και ο συγχρονισμός της κατάστασης του cluster βασίζονται στη

φιλοσοφία του GitOps, όπου το αποθετήριο Git αποτελεί τη μοναδική πηγή αλήθειας (Single Source of Truth) για ολόκληρο το σύστημα.

3.2.1. Continuous Integration (CI) μέσω Jenkins

Η διαδικασία Continuous Integration υλοποιείται μέσω της πλατφόρμας Jenkins, η οποία εκτελείται εντός του cluster σε διακριτό namespace (jenkins). Η λογική της αυτοματοποιημένης μεταγλώττισης και κατασκευής (build) ορίζεται στο κεντρικό αρχείο Jenkinsfile στη ρίζα του Monorepo.

Το Jenkins παρακολουθεί το Git Repository και, με την ανίχνευση κάθε νέου commit, εκκινεί αυτόματα το pipeline.

Αναλαμβάνει την κατασκευή των Container Images (Docker builds) για τα microservices της εφαρμογής και στη συνέχεια πραγματοποιεί την ασφαλή προώθησή τους (push) στο κεντρικό Container Registry (Docker Hub).

Λόγω της Stateless φύσης του deployment του Jenkins, σε περιπτώσεις επανεκκίνησης του Pod, η αρχική κατάσταση ανακτάται άμεσα μέσω του Pipeline script από το SCM (Source Control Management), ενώ η αυθεντικοποίηση και η πρώτη πρόσβαση του διαχειριστή πραγματοποιούνται με την ασφαλή ανάκτηση του κρυπτογραφημένου κλειδιού (initialAdminPassword) απευθείας από τα logs του Kubernetes Control Plane.

3.2.2. Continuous Deployment (CD) μέσω ArgoCD

Για τη διαδικασία του Continuous Delivery και Deployment χρησιμοποιείται το εργαλείο ArgoCD, το οποίο εφαρμόζει υποδειγματικά τις GitOps πρακτικές σε επίπεδο cluster.

Το ArgoCD συνδέεται απευθείας με τον κατάλογο των deployment manifests στο Git Repo και παρακολουθεί συνεχώς αν η επιθυμητή κατάσταση (Desired State), όπως αυτή περιγράφεται στα declarative YAML αρχεία, συμπίπτει με την πραγματική κατάσταση (Actual State) που εκτελείται εντός του MicroK8s cluster.

Σε περίπτωση που εντοπιστεί οποιαδήποτε τροποποίηση στο Git (π.χ. αναβάθμιση της έκδοσης ενός image από το Jenkins) ή χειροκίνητη αλλοίωση των ρυθμίσεων εντός του cluster (configuration drift), το ArgoCD εκκινεί αυτόματα τη διαδικασία συγχρονισμού (Automated Sync).

Με αυτόν τον τρόπο, εφαρμόζει άμεσα τα νέα manifests, διασφαλίζοντας ότι η υποδομή παραμένει πάντα ευθυγραμμισμένη με τον κώδικα του αποθετηρίου, αυξάνοντας την αξιοπιστία και την ταχύτητα διάθεσης της εφαρμογής.

4. Serverless Επεξεργασία (Serverless Computation)

4.1. Θεωρητικό Υπόβαθρο & Αρχιτεκτονική Event-Driven

Στο πλαίσιο της μετάβασης της πλατφόρμας σε Cloud Native πρότυπα, κρίθηκε απαραίτητη η εισαγωγή του μοντέλου Serverless Computation για διεργασίες που εκτελούνται αποκλειστικά ως απόκριση σε συγκεκριμένα συμβάντα (Event-Driven Architecture). Η τεχνολογία αυτή επιτρέπει στα compute components να κλιμακώνουν δυναμικά (Scale-to-Zero) και να δεσμεύουν πόρους μόνο κατά τη διάρκεια εκτέλεσης της διεργασίας, ελαχιστοποιώντας την άσκοπη κατανάλωση μνήμης και επεξεργαστικής ισχύος.

Στο παρόν σενάριο χρήσης, η Serverless αρχιτεκτονική επιλέχθηκε για την αυτοματοποιημένη εξαγωγή, ανάλυση και προσθήκη μεταδεδομένων (metadata annotation) στα JSON αρχεία των συναγερμών (alarms), αμέσως μετά την αποθήκευσή τους στο MinIO Object Storage.

4.2. Μεθοδολογία Υλοποίησης, Προκλήσεις και Εναλλακτική Προσέγγιση

Η αρχική σχεδιαστική προσέγγιση προέβλεπε την πλήρη εγκατάσταση και παραμετροποίηση του οικοσυστήματος Knative εντός του τοπικού MicroK8s cluster, με σκοπό τη δημιουργία ενός αυτόνομου Knative Service. Το Service αυτό θα ενεργοποιούνταν μέσω S3 Webhook triggers (Event Notifications) που θα απέστελλε το MinIO Object Storage κάθε φορά που πραγματοποιούνταν ένα νέο upload αρχείου alarm.

Ωστόσο, κατά τη φάση της υλοποίησης και της live ενσωμάτωσης του Knative, ανακύψαν σημαντικά τεχνικά προβλήματα και περιορισμοί στο περιβάλλον του cluster, τα οποία κατέστησαν αδύνατη την πλήρη και σταθερή λειτουργία του serverless runtime σε πραγματικό χρόνο.

Ως εκ τούτου, εφαρμόστηκε μια εναλλακτική αρχιτεκτονική λύση (Fallback Strategy) προκειμένου να μην αλλοιωθεί η επιχειρηματική λογική του σεναρίου χρήσης:

⇒ *Declarative Ανάπτυξη Κώδικα (Infrastructure-Ready)*: Η Serverless Function αναπτύχθηκε σε περιβάλλον Python (serverless_function.py) και εντάχθηκε σε διακριτό κατάλογο (serverless/) εντός του κεντρικού Git Monorepo. Με αυτόν τον τρόπο, η συνάρτηση είναι πλήρως διαθέσιμη σε επίπεδο πηγαίου κώδικα και έτοιμη προς άμεσο deploy (Deployment-Ready) σε οποιοδήποτε παραγωγικό Cloud περιβάλλον (π.χ. AWS Lambda ή Knative Production Mesh).

⇒ *Προσομοίωση Ροής (Event Choreography)*: Η event-driven λογική και η επεξεργασία των μεταδεδομένων ενσωματώθηκαν εναλλακτικά στη ροή του Node-RED. Αμέσως μετά την επιτυχή αποθήκευση του αρχείου στο MinIO Node, η ροή εκτελεί ένα ασύγχρονο HTTP POST Request, το οποίο

μεταφέρει το payload του συμβάντος, εκτελεί τη λογική της συνάρτησης για την παραγωγή των metadata και επιστρέφει το εμπλουτισμένο JSON, επιτυγχάνοντας το επιθυμητό αρχιτεκτονικό αποτέλεσμα χωρίς τη χρήση του Knative runtime.

5. Δοκιμές Φόρτου, Πειραματικές Μετρήσεις & SLAs (Service Level Agreements)

5.1. Μεθοδολογία Πειραματικών Δοκιμών (Load Testing)

Για την αντικειμενική αξιολόγηση της ποιότητας υπηρεσίας (Quality of Service - QoS) και της σταθερότητας της Cloud Native υποδομής υπό συνθήκες υψηλού φόρτου, σχεδιάστηκε και εκτελέστηκε ένα εξειδικευμένο σενάριο δοκιμών φόρτου (Load Testing) μέσω αυτοματοποιημένου script σε Python (load_test.py).

Το script προσομοίωσε την ταυτόχρονη και μαζική αποστολή αιτημάτων τηλεμετρίας και παραγωγής κρίσιμων συναγερμών (alarms) από τις 15 IoT συσκευές της πλατφόρμας. Κατά τη διάρκεια των δοκιμών, μετρήθηκε με ακρίβεια χιλιοστού του δευτερολέπτου (ms) ο συνολικός χρόνος απόκρισης (Round-Trip Time - RTT) της αλυσίδας επεξεργασίας: από τη στιγμή της εισαγωγής του alarm στο ThingsBoard, τη δρομολόγηση μέσω του RabbitMQ, την επεξεργασία στο Node-RED, έως την τελική αποθήκευση του δομημένου αρχείου JSON στο MinIO Object Storage.

5.2. Στατιστική Ανάλυση & Αθροιστική Συνάρτηση Κατανομής (CDF Percentiles)

Τα αποτελέσματα των μετρήσεων υποβλήθηκαν σε στατιστική ανάλυση με τη χρήση της Αθροιστικής Συνάρτησης Κατανομής (Cumulative Distribution Function - CDF), εστιάζοντας στους δείκτες 95th και 99th Percentiles. Η προσέγγιση αυτή επιτρέπει την απομόνωση των ακραίων τιμών καθυστέρησης (network or cold-start spikes) και αποτυπώνει την πραγματική συμπεριφορά του συστήματος στο 95% και 99% των παραγωγικών αιτημάτων.

Τα πειραματικά δεδομένα έδειξαν εξαιρετική σταθερότητα και υψηλό throughput, με τη ροή των δεδομένων να επιτυγχάνει Request Success Rate της τάξης του 100%, χωρίς απώλειες πακέτων ή blocking φαινόμενα στις ουρές του Message Broker. Συγκεκριμένα, οι χρόνοι απόκρισης διαμορφώθηκαν ως εξής:

- ✓ *Διάμεσος Χρόνος (Median / 50th Percentile):* 9 ms
- ✓ *95th Percentile (95% CDF):* 13 ms (σημαίνει ότι το 95% των αιτημάτων εξυπηρετήθηκε σε χρόνο ίσο ή μικρότερο των 13ms)
- ✓ *99th Percentile (99% CDF):* 20 ms (σημαίνει ότι το 99% των αιτημάτων ολοκλήρωσε την εγγραφή στο Object Storage σε λιγότερο από 20ms)

5.3. Καθορισμός Κλάσεων Ποιότητας Υπηρεσίας (SLA Levels)

Με βάση τα ανωτέρω στατιστικά συμπεράσματα των πειραματικών μετρήσεων, θεσπίστηκαν τρεις διακριτές κλάσεις ποιότητας υπηρεσίας (SLA Levels) για την πλατφόρμα Logistics Καυσίμων, ανάλογα με την κρισιμότητα των συσκευών και των παραγόμενων συμβάντων:

1) Χρυσή Κλάση (Gold Class - Real-Time Critical Alarms):

- *Προδιαγραφή SLA:* Χρόνος απόκρισης ≤ 13 ms (βασισμένος στο 95% CDF).

- *Εφαρμογή:* Ενεργοποιείται αποκλειστικά για τις κρίσιμες συσκευές υψηλής επικινδυνότητας, όπως τα βυτιοφόρα εν κινήσει (diesel truck, gas truck) και τις σταθερές δεξαμενές καυσίμων (diesel tank, gas tank), όπου οι συναγερμοί υπερθέρμανσης, πίεσης ή κρίσιμης στάθμης απαιτούν ακαριαία δρομολόγηση.

2) Αργυρή Κλάση (Silver Class - Standard Monitoring):

- *Προδιαγραφή SLA:* Χρόνος απόκρισης μεταξύ 14 ms και 20 ms (βασισμένος στο 99% CDF).

- *Εφαρμογή:* Αφορά την τυπική ροή συνεχών δεδομένων τηλεμετρίας (όχι alarms) και την επιτήρηση των σταθερών υποδομών περιβάλλοντος, όπως οι κεντρικές αποθήκες ελαιολιπαντικών (warehouse).

3) Χάλκινη Κλάση (Bronze Class - Batch & Historical Analysis):

- *Προδιαγραφή SLA:* Χρόνος απόκρισης > 20 ms.

- *Εφαρμογή:* Εφαρμόζεται για δευτερεύουσες διεργασίες χαμηλής προτεραιότητας, όπως η batch αρχειοθέτηση παλαιών logs, η εξαγωγή αναφορών και η εκτέλεση της Serverless function για τον υπολογισμό δευτερευόντων μεταδεδομένων ιστορικού.

Η θέσπιση των συγκεκριμένων επιπέδων SLA διασφαλίζει ότι η cloud πλατφόρμα μπορεί να εγγυηθεί την απαιτούμενη ποιότητα υπηρεσίας, κατανέμοντας ορθά την προτεραιότητα των υπολογιστικών πόρων του Kubernetes cluster ανάλογα με την επιχειρησιακή ανάγκη κάθε microservice.

6. Περιορισμοί Υλοποίησης & Μελλοντικές Βελτιώσεις (Future Work)

6.1. Υφιστάμενοι Περιορισμοί του Πρωτοτύπου

Η τρέχουσα υλοποίηση της πλατφόρμας αποτελεί ένα πλήρως λειτουργικό ακαδημαϊκό πρωτότυπο (Proof of Concept - PoC), το οποίο αξιολογεί επιτυχώς τη ροή δεδομένων, τη GitOps φιλοσοφία και τη συμπεριφορά του συστήματος υπό ελεγχόμενο φόρτο. Ωστόσο, η ανάπτυξη της εφαρμογής σε περιβάλλον τοπικού single-node cluster (MicroK8s σε τοπικό host) εισάγει συγκεκριμένους περιορισμούς:

- **Hardware Constraints:** Η συνύπαρξη πολλαπλών βαρέων microservices (ThingsBoard, RabbitMQ, ArgoCD, Jenkins) στον ίδιο φυσικό πόρο περιορίσει τη δυνατότητα live εκτέλεσης των αυτοματοποιήσεων και του Serverless runtime, λόγω της αυξημένης κατανάλωσης μνήμης RAM και CPU Throttling.

- **Ephemeral Storage:** Αν και τα microservices ακολουθούν το Stateless Pattern, η απουσία ενός κατανεμημένου Production Storage Class (π.χ. Ceph, Rook ή AWS EBS) καθιστά τα δεδομένα του ThingsBoard και του RabbitMQ εφήμερα σε περίπτωση ολικής κατάρρευσης του node.

6.2. Μελλοντικές Επεκτάσεις για μια Παραγωγική Cloud Native Εφαρμογή (Production-Ready)

Προκειμένου η πλατφόρμα Logistics Καυσίμων να μετασχηματιστεί σε μια πλήρως βιομηχανική, ανθεκτική και high-available cloud native εφαρμογή, θα μπορούσαν να γίνουν οι ακόλουθες μελλοντικές βελτιώσεις:

6.2.1. Live Ενσωμάτωση Multi-Cloud IaC & Serverless Runtimes

Κανονική Εγκατάσταση OpenTofu & Ansible: Στόχος είναι η ζωντανή εκτέλεση του καταλόγου *automation/* σε ένα πλήρως κατανεμημένο περιβάλλον. Το OpenTofu θα αναλάβει το provisioning ενός Multi-Node Managed Kubernetes Cluster (π.χ. AWS EKS) σε πολλαπλές ζώνες διαθεσιμότητας (Multi-AZ), και το Ansible θα αυτοματοποιήσει τις ρυθμίσεις ασφαλείας (OS hardening, IAM roles, SSH keys) των nodes.

Πλήρης Knative Integration: Μεταφορά της `serverless_function.py` στο επίσημο Knative Serving / Eventing framework του cloud, συνδέοντας τα native S3 buckets του MinIO μέσω CloudEvents. Αυτό θα επιτρέψει τον real-time υπολογισμό metadata με πλήρη υποστήριξη Scale-to-Zero, εκμηδενίζοντας το κόστος computation όταν δεν παράγονται alarms.

6.2.2. Διαχείριση Κατάστασης & Ασφάλειας (State & Secret Management)

Μετάβαση σε StatefulSets με Persistent Volumes (PV): Για τα microservices που διατηρούν κρίσιμη κατάσταση (ThingsBoard DB, RabbitMQ Queues),

προβλέπεται η μετατροπή των Deployments σε Kubernetes StatefulSets. Αυτά θα συνδέονται με Persistent Volume Claims (PVCs) σε cloud block storages, διασφαλίζοντας ότι κανένα μήνυμα ή alarm δεν θα χαθεί ακόμα και σε περίπτωση καταστροφής ολόκληρου του pod.

Εξωτερική Διαχείριση Μυστικών (External Secret Management): Αντικατάσταση των default Kubernetes Secrets με μια ολοκληρωμένη λύση Enterprise Secret Management, όπως το HashiCorp Vault. Τα ευαίσθητα δεδομένα (Access Tokens των 15 συσκευών, κωδικοί MinIO, initialAdminPassword του Jenkins) θα κρυπτογραφούνται εξωτερικά και θα εισάγονται δυναμικά στα pods κατά το runtime (runtime injection), αποτρέποντας την έκθεσή τους στο Git.

6.2.3. Παρακολούθηση Μετρικών & Αυτόματη Κλιμάκωση (Observability & Autoscaling)

Εγκατάσταση Prometheus & Grafana: Ενσωμάτωση των cloud native εργαλείων Observability για την πλήρη παρακολούθηση του cluster. Θα συλλέγονται τουλάχιστον δύο βασικές μετρικές εφαρμογής: η χρήση μνήμης (Memory Utilization) του Node-RED και το μέγεθος της ουράς μηνυμάτων (Queue Depth) του RabbitMQ.

Σύνδεση με Horizontal Pod Autoscaler (HPA): Σύνδεση των παραπάνω μετρικών με τον Keda (Kubernetes Event-driven Autoscaling) ή τον HPA. Για παράδειγμα, αν ο ρυθμός των alarms αυξηθεί απότομα κατά τη διάρκεια ενός load test και η ουρά του RabbitMQ ξεπεράσει τα 500 εκκρεμή μηνύματα, το Kubernetes θα προχωρά αυτόματα σε οριζόντια κλιμάκωση (scale-out), αυξάνοντας τα Node-RED Pods από 1 σε 5 για την ταχύτερη αποσυμφόρηση του συστήματος και τη διατήρηση του Gold SLA.